MCMP117 · WEEK 3

Fundamentals of Operating Systems

Week 3 · OS Services, System Calls & Processes

What does the OS do for a program — and what exactly is a running program?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 2 in Four Ideas

Two views of the OS

An extended machine (clean abstractions) and a resource manager (sharing the hardware).

Dual mode

A mode bit in the CPU: user mode is restricted, kernel mode may do everything.

A first system call

printf() ended up as write(1, …); strace showed us every request to the kernel.

Calls are expensive

On our test machine a getpid() system call cost ≈ 116 ns, about 76 × a function call.

Today: which services sit behind that system-call door, how a call actually travels into the kernel — and what a "process" is.

Today's Agenda

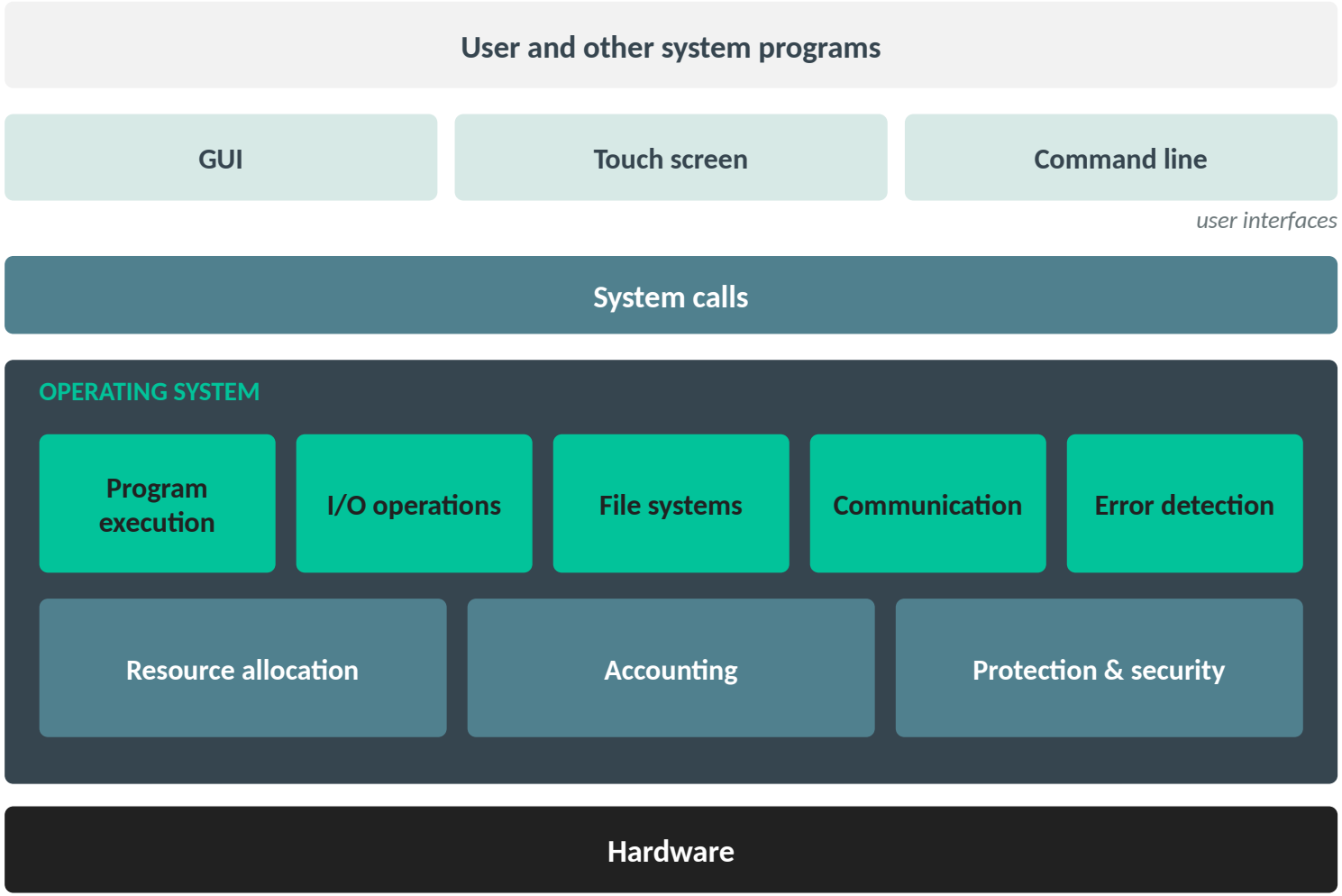
1. **OS services** What the OS offers to users, to programs, and to the system itself.
2. **System calls** API vs. system call, the system-call table, the trap, passing parameters.
3. **Processes** Program vs. process, memory layout, the five states, the PCB.
4. **Context switch** What is saved, why it is pure overhead, and how much it costs.
5. **Hands-on** C programs: raw syscall, memory map, fork / exec / wait, fork trees.

PART 1

Operating-System Services

What the OS provides — to you, to your programs, and to itself.

A View of Operating-System Services



Two groups

- **Mint row:** services that help the user and the user's programs get work done.
- **Slate row:** services that keep the system itself efficient and safe when many users and programs share it.
- Programs reach every service through the same door: system calls.

SERVICES

Services That Help the User

User interface

Command line (bash, PowerShell), graphical desktop, touch screen — or a batch file of commands.

Program execution

Load a program into memory, run it, and end it — normally or with an error code.

I/O operations

Programs never drive devices directly; the OS does the I/O for them (Week 2: dual mode).

File-system manipulation

Create, delete, read, write and search files and directories; manage permissions.

Communication

Exchange data between processes: on one computer (pipes, shared memory) or across a network (sockets).

Error detection

Notice and handle errors: a disk read fails, the printer is out of paper, a program divides by zero.

SERVICES

Services That Keep the System Running Well

With many users and programs on one machine, the OS also needs services for itself.

Resource allocation

Who gets the CPU, how much memory, which printer? Done by schedulers and allocators (Weeks 4–7).

Linux: ulimit -a shows per-process limits.

Accounting & logging

Keep track of who used how much CPU time, memory and disk — for billing, statistics and tuning.

Linux: ps, top, journalctl; Windows: Event Viewer.

Protection & security

Protection: processes cannot touch each other's memory. Security: users must authenticate; attacks are logged.

Linux: file permissions (Week 11), passwords.

A chain is only as strong as its weakest link: one guessed password defeats all of the protection hardware we met last week.

Command Line vs. Graphical Interface

```
# a command-line session (bash on Linux)
$ ls -l report.txt      # file-system service
$ cp report.txt backup/ # file-system service
$ ./myprog &            # program execution
$ ps                    # status information
$ kill 4242             # process control

# each command is a separate program that
# the shell starts for you (Part 3: fork/exec)
```

The shell is not part of the kernel: it is an ordinary program that reads a command and asks the OS to run it.

Which one is better?

- **Command line (CLI):** fast for experts, scriptable, works over a slow network connection; must remember commands.
- **Graphical (GUI):** windows, icons, mouse; easy to learn; hard to automate.
- **Touch screen:** gestures instead of a mouse; phones and tablets.
- Most system administrators use both. From Week 9 on we work in the Linux shell.

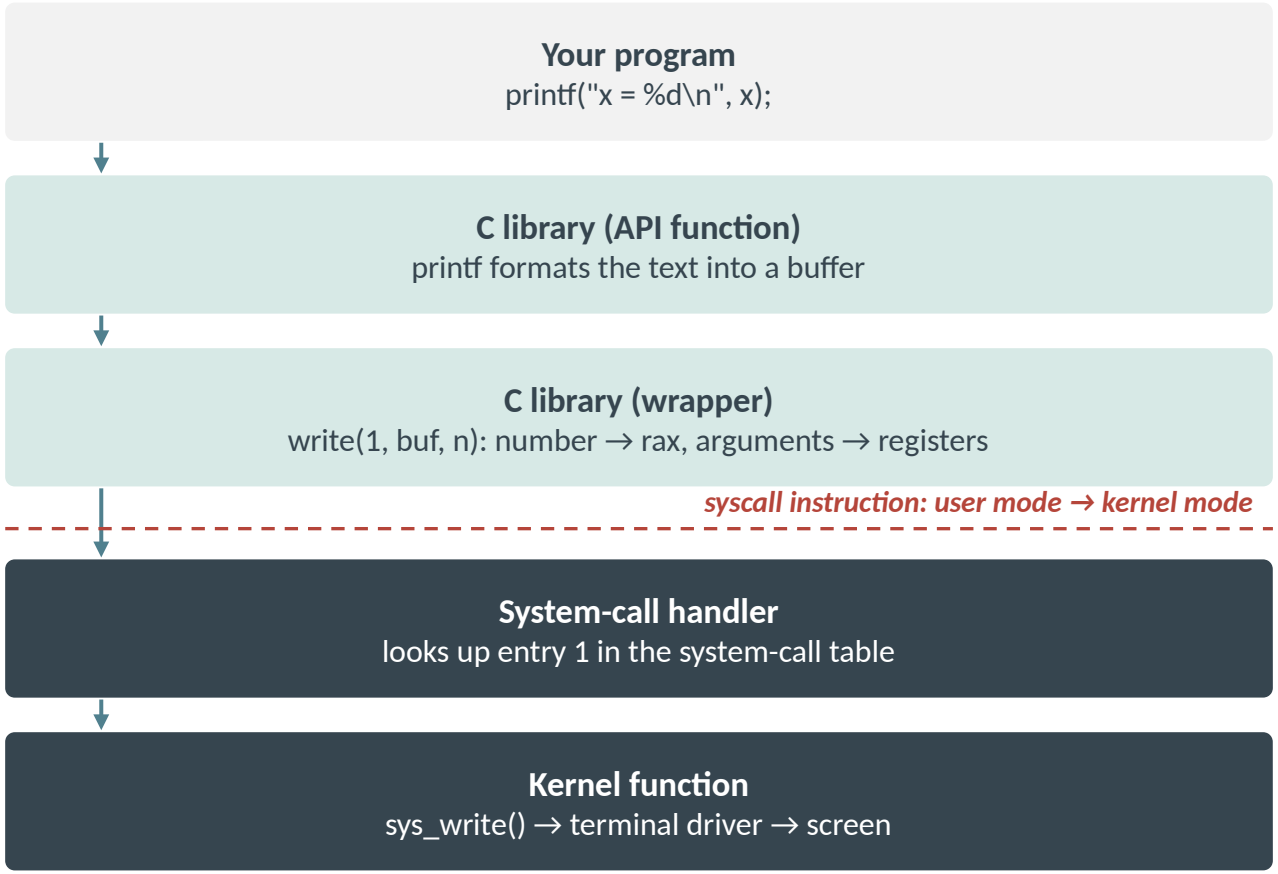
PART 2

System Calls in Detail

The only door from a user program into the kernel.

API, Library Wrapper and System Call

Programmers almost never execute a system call directly. Three layers sit between printf() and the hardware.



Three words, three meanings

- **API** (application programming interface): the documented functions a programmer calls, e.g. POSIX or the Windows API.
- **Wrapper:** a tiny library function with the same name as the system call; it only loads registers and traps.
- **System call:** the actual entry into the kernel.
- Not every API function makes a system call: strlen() and sqrt() never do; printf() only when its buffer is flushed.

Why Program Against an API?

Three widely used APIs — each one hides the real system calls of its platform.

POSIX API

- Linux, macOS, the BSDs.
- Provided by the C library (glibc, musl, ...).
- write(), fork(), open()...

Windows API

- Windows.
- CreateProcess(), ReadFile()...
- Internally calls ntdll.dll, which makes the real kernel calls (NtCreateUserProcess...).

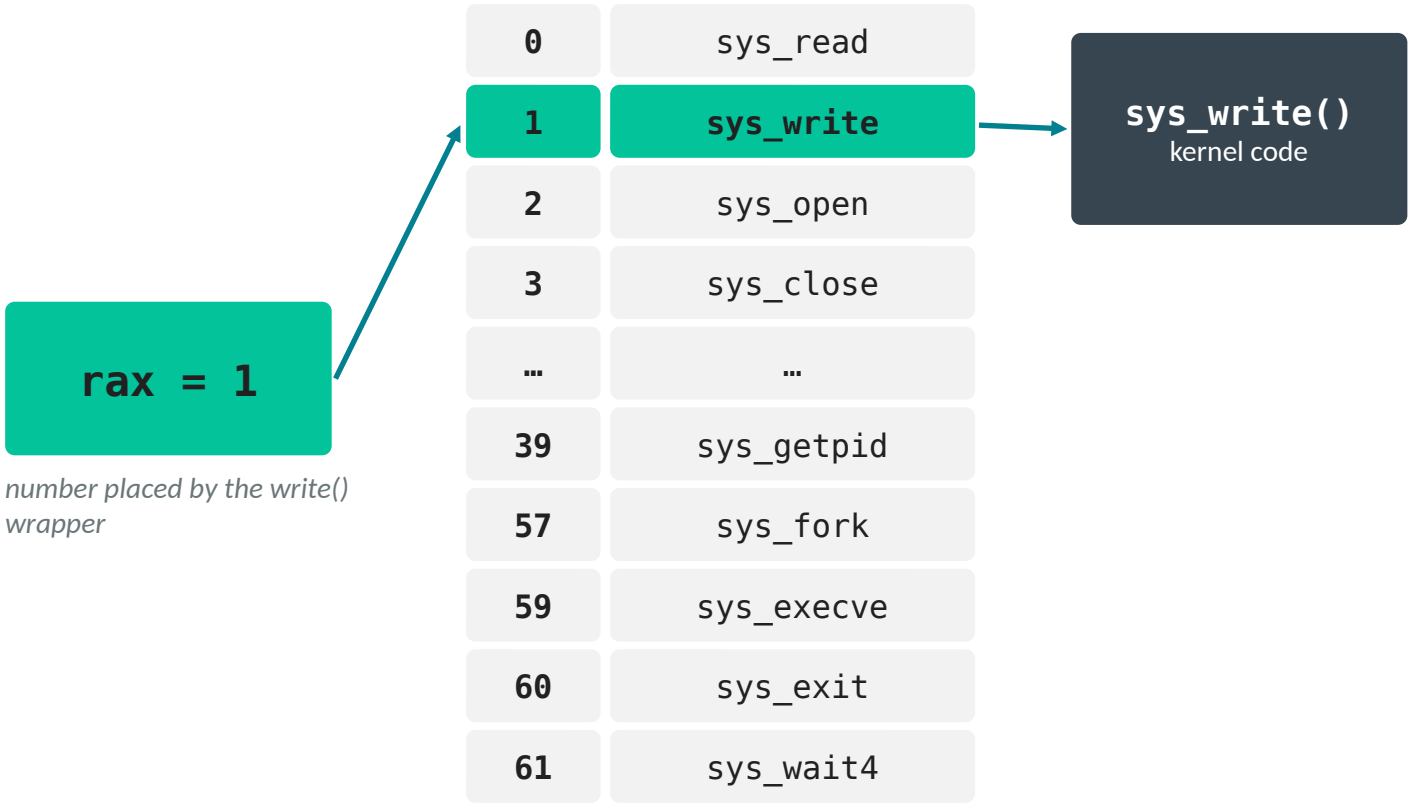
Java API

- Any OS with a Java Virtual Machine.
- Files.readAllBytes(), ProcessBuilder...
- The JVM translates them into the host OS's system calls.

Portability: write is system call number 1 on 64-bit x86 Linux, 4 on 32-bit x86 Linux and 64 on ARM64 Linux. The API name write() stays the same everywhere.

Every System Call Has a Number

The kernel keeps a table: system-call number → address of the kernel function that does the work.

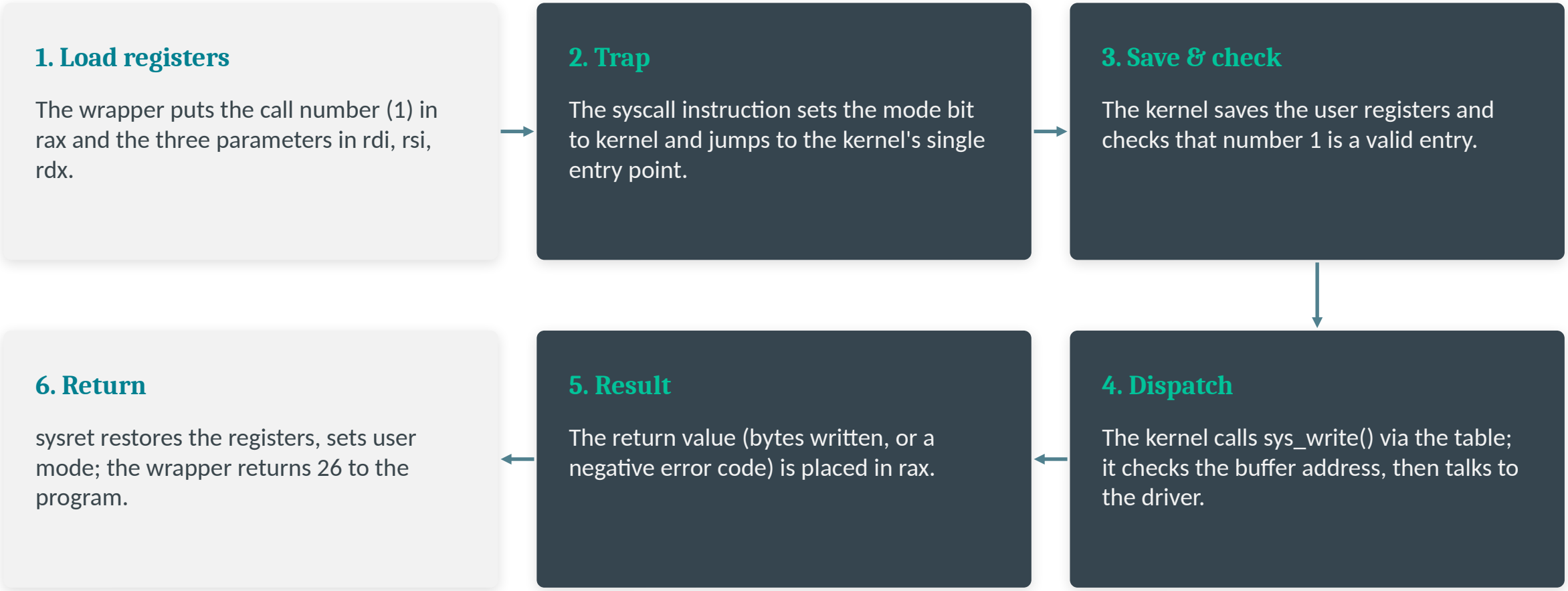


system-call table (x86-64 Linux)

Why a table?

- A user program may enter the kernel only at one fixed entry point — never jump to an arbitrary kernel address.
- The number selects the service; the kernel checks that it is valid, then calls the function from the table.
- Linux 6.8 on x86-64 has about 370 system calls, numbered 0 to 461.

One write(): The Trip into the Kernel, Step by Step



light = user mode dark = kernel mode

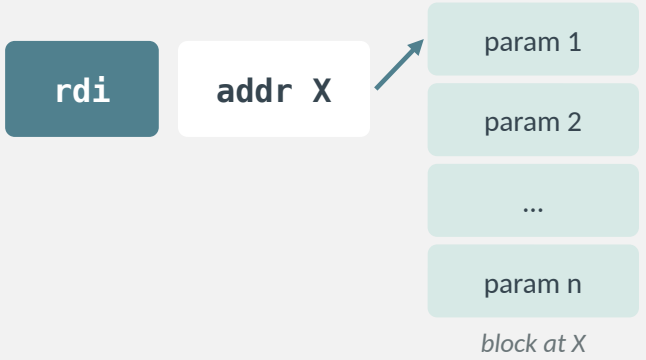
Three Ways to Pass Parameters to the OS

1. In registers



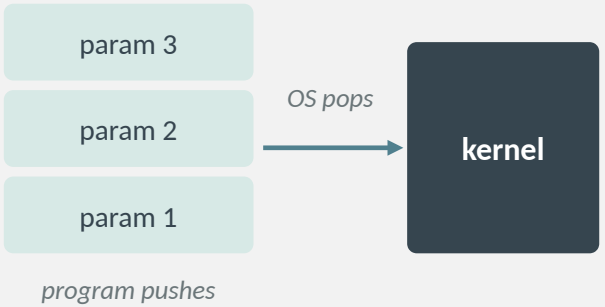
Fastest. Limited by the number of registers (6 on x86-64 Linux).

2. In a memory block



Any number of parameters; only the address travels in a register. Example: clone3() gets a pointer to a struct.

3. On the stack



Flexible, but the kernel must copy from user memory. Example: 32-bit FreeBSD on x86.

Linux combines 1 and 2: up to six parameters in registers; larger data (a buffer, a structure) is passed as an address in a register.

Concrete Example: x86-64 Linux Register Convention

Register	Role in a system call	For write(1, msg, 26)
rax	system-call number (in) · return value (out)	1 → 26 (bytes written)
rdi	1st parameter	1 (file descriptor: standard output)
rsi	2nd parameter	address of msg
rdx	3rd parameter	26 (number of bytes)
r10, r8, r9	4th, 5th, 6th parameter	— (not used by write)
rcx, r11	overwritten by the syscall instruction	the CPU saves the return address and flags here

A return value between −4095 and −1 means an error: the wrapper stores the error number in errno and returns −1 to your program.

HANDS-ON

Example 1: The Same Message, Three Levels Deep

```c
int main(void)
{
    printf("1) printf  -> libc -> write\n");     /* level 1 */
    fflush(stdout);

    /* level 2 */
    if (write(1, "2) write() -> libc wrapper\n", 27) != 27)
        return 1;

    /* level 3: do what the wrapper does, by hand.
     *   rax = 1   system-call number of write (x86-64)
     *   rdi = 1   1st parameter: file descriptor (stdout)
     *   rsi = msg 2nd parameter: address of the bytes
     *   rdx = 26  3rd parameter: number of bytes
     * The 'syscall' instruction traps into the kernel; the
     * kernel's return value comes back in rax.         */
    static const char msg[] = "3) syscall instruction!!!\n";
    long ret;
    __asm__ volatile ("syscall"
                      : "=a"(ret)                /* out: rax */
                      : "a"(1L), "D"(1L), "S"(msg), "d"(26L)
                      : "rcx", "r11", "memory"); /* clobbered */
    printf("kernel returned %ld (bytes written)\n", ret);
    return 0;
}
```

What to observe

- Level 3 does by hand what the write() wrapper does: number in rax, parameters in rdi, rsi, rdx, then syscall.
- strace cannot tell the three levels apart: to the kernel they are identical.

```
$ strace ./raw_write
write(1, "1) printf  -> libc -> write\n", 28) = 28
write(1, "2) write() -> libc wrapper\n", 27) = 27
write(1, "3) syscall instruction!!!\n", 26) = 26
write(1, "kernel returned 26 (bytes writte"..., 35) = 35
exit_group(0)                           = ?
```

TYPES

Six Categories of System Calls

Category	Typical requests	Linux (POSIX)	Windows API
Process control	create, end, load, wait	fork(), execve(), exit(), wait4()	CreateProcess(), ExitProcess(), WaitForSingleObject()
File management	create, open, read, write, close	open(), read(), write(), close()	CreateFile(), ReadFile(), WriteFile(), CloseHandle()
Device management	request, read, write, configure	ioctl(), read(), write()	SetConsoleMode(), ReadConsole(), WriteConsole()
Information maintenance	time, date, IDs, system data	getpid(), uname(), clock_gettime()	GetCurrentProcessId(), GetSystemTime()
Communication	pipes, shared memory, network	pipe(), mmap(), socket()	CreatePipe(), MapViewOfFile()
Protection	permissions, ownership	chmod(), chown(), umask()	SetFileSecurity(), SetSecurityDescriptorGroup()

WORKED EXAMPLE

How Many System Calls Does Copying a File Take?

The classic sequence

- Ask for the two file names (I/O).
- Open the input file; on error print a message and exit (file management, process control).
- Create the output file.
- Loop: read a block, write a block — until read() returns 0 (end of file).
- Close both files, report success, exit.
- Every step is one or more system calls — and so is loading the program itself.

```
$ strace -c cp in.bin out.bin   # in.bin = 1 MiB
 calls  syscall            (trimmed)
    26  mmap
    13  close
    11  openat
    10  fstat
     9  read
     2  copy_file_range
   102  total
```

Only 2 of 102 calls move the data: modern cp asks the kernel to copy inside the kernel. Most calls load the program and its libraries.

System Programs: The OS as Most Users See It

Programs shipped with the OS that make system calls on your behalf. They run in user mode.

File management

ls, cp, mv, rm, mkdir; File Explorer, Finder.

Status information

date, df, free, ps, top; Task Manager; system logs.

File modification

Text editors (nano, vim, Notepad); search tools (grep, find).

Programming support

Compilers, assemblers, debuggers, interpreters: gcc, gdb, python.

Loading & execution

Loaders and linkers that place programs and libraries in memory.

Background services

Daemons / services: sshd, cron, the print spooler. Started at boot.

INTERMISSION

10-Minute Break

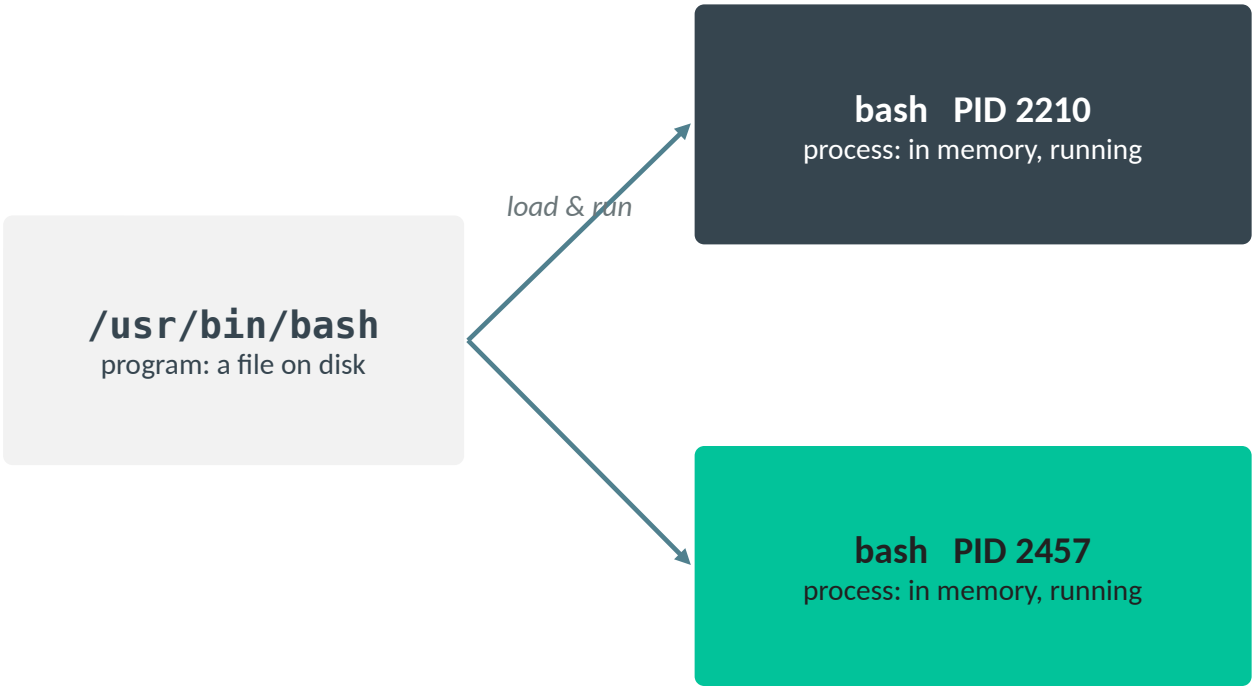
Next: the process — the unit of work the OS manages.

PART 3

The Process Concept

A program in execution.

Program vs. Process



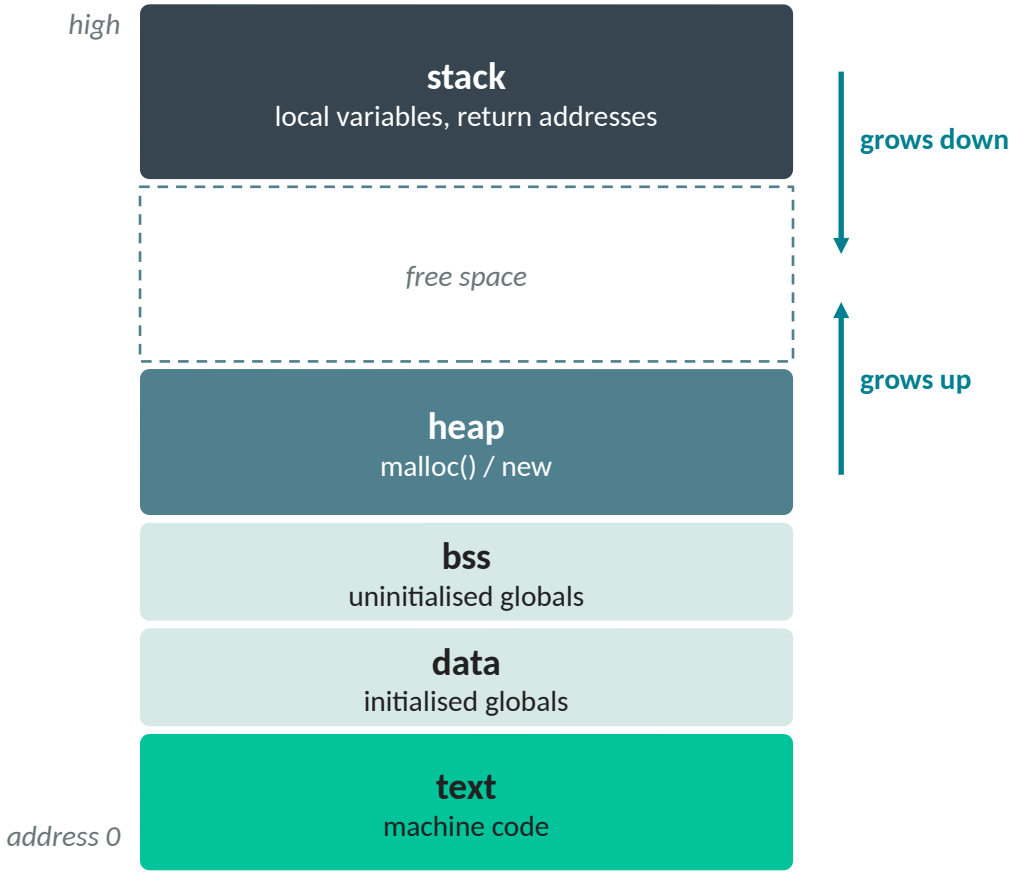
one program, two processes (two terminal windows)

Definitions

- **Program:** passive — instructions stored in an executable file.
- **Process:** active — a program in execution: its code plus the current activity (program counter, registers), its memory and its open files.
- A program becomes a process when it is loaded into memory.

Analogy: a recipe is a program; cooking it is a process. Two cooks can follow one recipe at the same time, each with their own ingredients and progress.

A Process in Memory: Five Regions



fixed size after loading: text, data, bss · grow and shrink while running: heap, stack

What lives where (in C)

- **text:** the compiled code of main() and every other function; read-only.
- **data:** int initialised = 42; (global with a value)
- **bss:** int uninitialised; (global without a value; the OS fills it with zeros)
- **heap:** memory from malloc(); lives until free().
- **stack:** int local = 7; inside a function; one frame per active call, removed when the function returns.
- Heap and stack grow towards each other into the free space between them.

HANDS-ON

Example 2: Printing the Address of Each Region

```c
#include <stdio.h>
#include <stdlib.h>

int initialised = 42;   /* global with a value    -> data */
int uninitialised;      /* global without a value -> bss  */

int main(void)
{
    int local = 7;                  /* local variable -> stack */
    int *dynamic = malloc(sizeof *dynamic);     /* malloc -> heap */

    printf("text  main()        %p\n", (void *)main);
    printf("data  initialised   %p\n", (void *)&initialised);
    printf("bss   uninitialised %p\n", (void *)&uninitialised);
    printf("heap  *dynamic      %p\n", (void *)dynamic);
    printf("stack local         %p\n", (void *)&local);

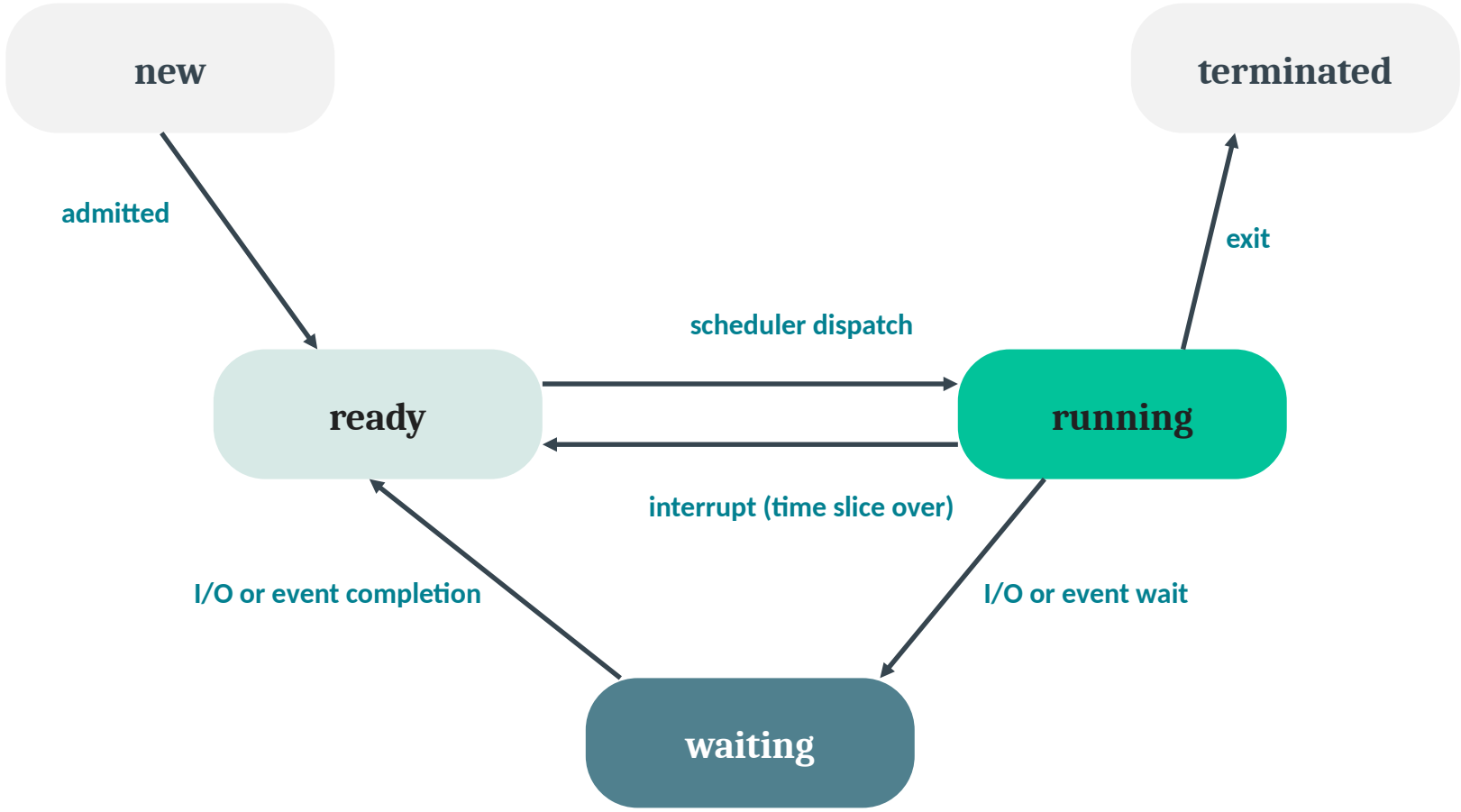
    free(dynamic);
    return 0;
}
```

What to observe

- Order: text < data < bss < heap, with the stack far above.
- Run it twice: every address changes. Linux places regions at random (ASLR) to make attacks harder.

```
$ ./mem_layout
text  main()        0x555d9ba160c0
data  initialised   0x555d9ba19010
bss   uninitialised 0x555d9ba19018
heap  *dynamic      0x555db649d2a0
stack local         0x7ffe2b4f6ab4
```

The Five-State Process Model



Rules

- One running process per CPU core.
- Many processes may be ready or waiting at once.
- Ready and waiting processes sit in queues: linked lists of their PCBs.
- In Linux ps: R = running or ready, S = waiting (sleeping), Z = zombie.

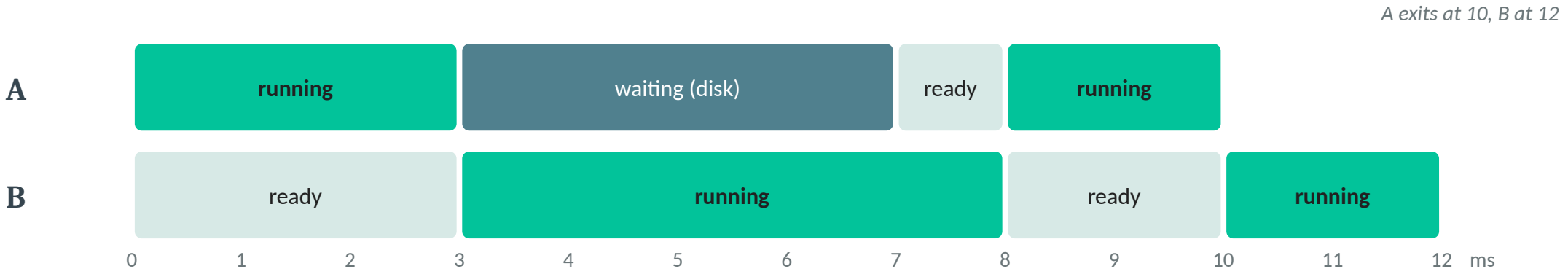
What Triggers Each Transition?

Transition	Triggered by	Everyday example
new → ready	the OS has created the PCB and loaded the program	you type ./mem_layout and press Enter
ready → running	the scheduler dispatches the process	the CPU core becomes free and the scheduler picks it
running → ready	an interrupt, usually the timer (time slice used up)	a long calculation is paused so others can run
running → waiting	a system call that must wait (I/O, child, sleep)	read() from the disk; wait() for a child
waiting → ready	the awaited event happens (often an interrupt)	the disk interrupt says the data has arrived
running → terminated	exit() or a fatal error	main() returns; the program divides by zero

Waiting → running is impossible: a woken process first joins the ready queue. Choosing who runs next is CPU scheduling (Weeks 4–5).

WORKED EXAMPLE

Two Processes, One CPU: States Over Time



Events

- **t = 3:** A calls read() → waiting; B is dispatched.
- **t = 7:** disk interrupt → A becomes ready (B keeps the CPU).
- **t = 8:** timer interrupt ends B's 5 ms slice → A runs.
- **t = 10:** A exits → B runs until it exits at 12.

```
CPU work needed: A = 3+2 = 5 ms
                 B = 5+2 = 7 ms

One after another (no overlap):
  A 9 ms (CPU idle 4) + B 7 ms = 16 ms
  CPU busy 12/16 = 75 %

Interleaved (above):
  12 ms, CPU busy 12/12 = 100 %
```

The PCB: Everything the OS Knows About a Process

process state

process ID (PID), parent PID

program counter

CPU registers

CPU-scheduling information

memory-management information

accounting information

I/O status: open files, devices

one PCB per process, kept in kernel memory

What the fields are for

- **State:** new, ready, running, waiting or terminated.
- **Program counter & registers:** where to continue, and the values to restore, when the process runs again.
- **Scheduling info:** priority, pointers into the scheduling queues.
- **Memory info:** where the process's regions are (page tables, limits; Weeks 6–7).
- **Accounting:** CPU time used, time limits, process numbers.
- **I/O status:** list of open files and allocated devices.
- In Linux the PCB is a C structure, struct task_struct, with hundreds of fields.

Peeking at a PCB: /proc/<pid>/status

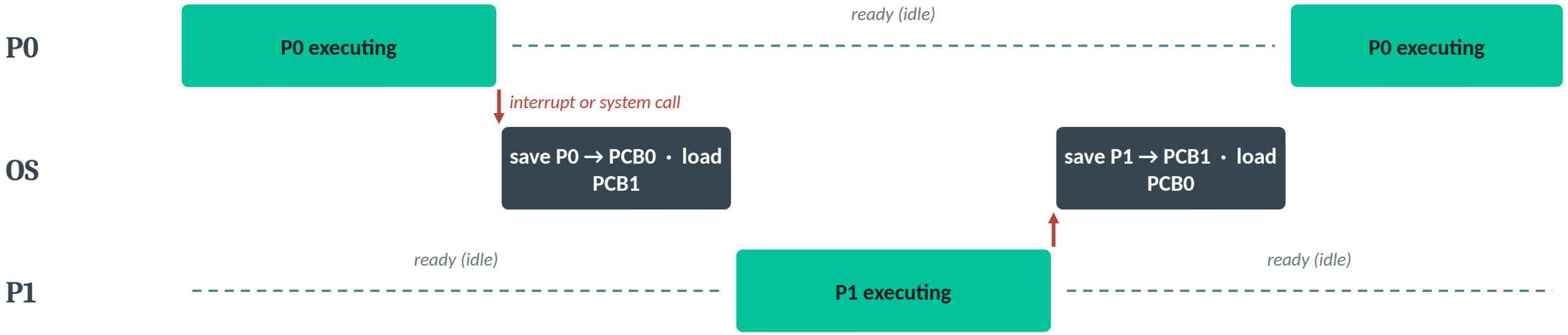
Linux shows parts of every process's PCB as a text file in the /proc directory.

```
$ sleep 30 &
$ grep -E "^(Name|State|Pid|PPid|Uid|VmSize|VmRSS|
  Threads|voluntary)" /proc/$!/status
Name:   sleep
State:  S (sleeping)
Pid:    27271
PPid:   27265
Uid:    0       0       0       0
VmSize:     2724 kB
VmRSS:      1696 kB
Threads:        1
voluntary_ctxt_switches:        2
nonvoluntary_ctxt_switches:     0
```

field → PCB entry

field	PCB entry
`State`	process state
`Pid, PPid`	identification
`Uid`	owner (protection)
`VmSize, VmRSS`	memory-management info
`Threads`	Week 4!
`…ctxt_switches`	accounting info

Context Switch: Moving the CPU from One Process to Another



What is saved and restored

Program counter, CPU registers, stack pointer, process state, and the memory-management information (which page table is active). All of it lives in the PCB.

During the dark boxes no process does useful work: a context switch is pure overhead.

WORKED EXAMPLE

How Much Does Context Switching Cost?

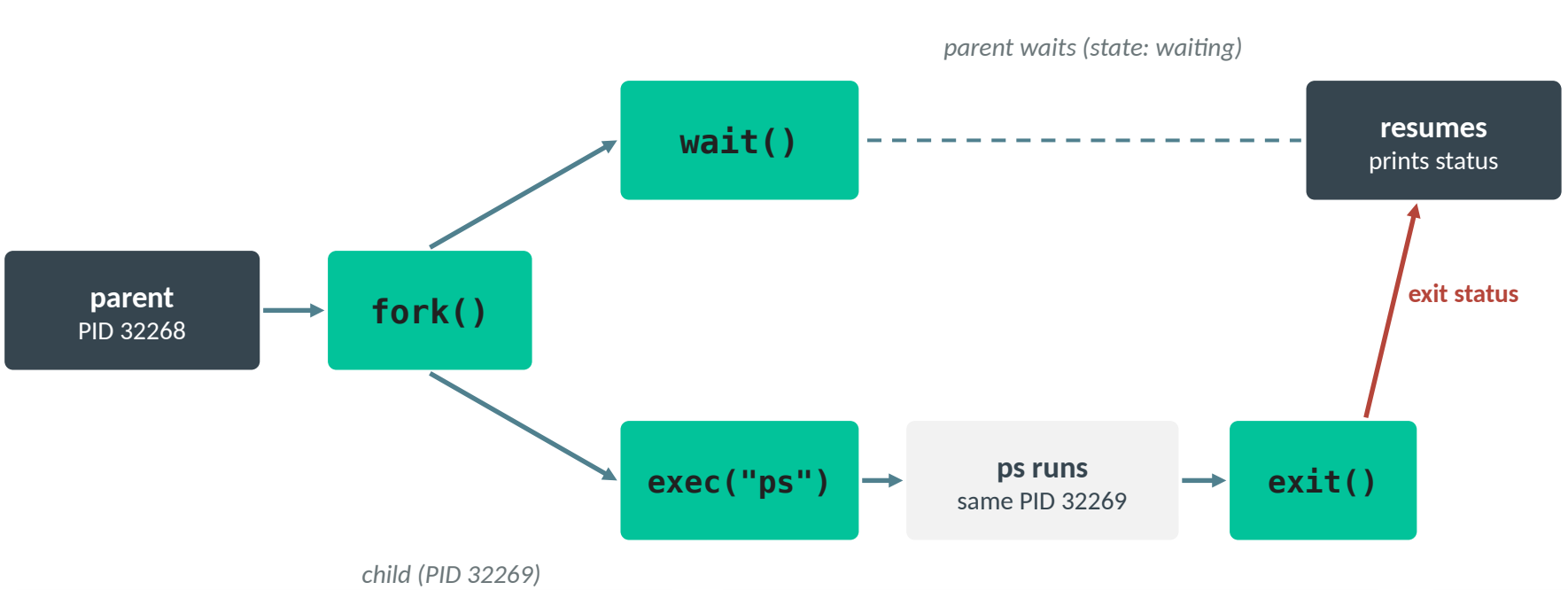
The model

- Each process runs for a time slice q, then the OS needs s to switch.
- Fraction of CPU time lost:
- overhead = s / (q + s)
- Assume s = 5 µs (illustrative; real switches cost a few µs, more when the caches must refill).
- Example: 1000 switches per second × 5 µs = 5 ms per second = 0.5 %.

```
time slice q    overhead s/(q+s)
100   ms         0.005 %
 10   ms         0.05  %
  1   ms         0.50  %
  0.1 ms         4.76  %
  0.01 ms       33.3   %
  0.005 ms      50     %   (q = s)
```

Short slices feel responsive but waste CPU; long slices waste little but feel sluggish. Choosing q is a Round-Robin question (Week 5).

Creating a Process: fork(), exec(), wait()



Windows does both steps in one call: CreateProcess().

fork()

Copies the calling process. Returns twice: 0 in the child, the child's PID in the parent (−1 on error).

exec()

Replaces the program in the process with a new one. The PID stays the same; it never returns on success.

wait()

Parent sleeps until a child ends, then collects its exit status.

HANDS-ON

Example 3: fork, exec and wait in C

```c
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);            /* empty the buffer BEFORE fork */

    pid_t pid = fork();        /* one process calls, two return */

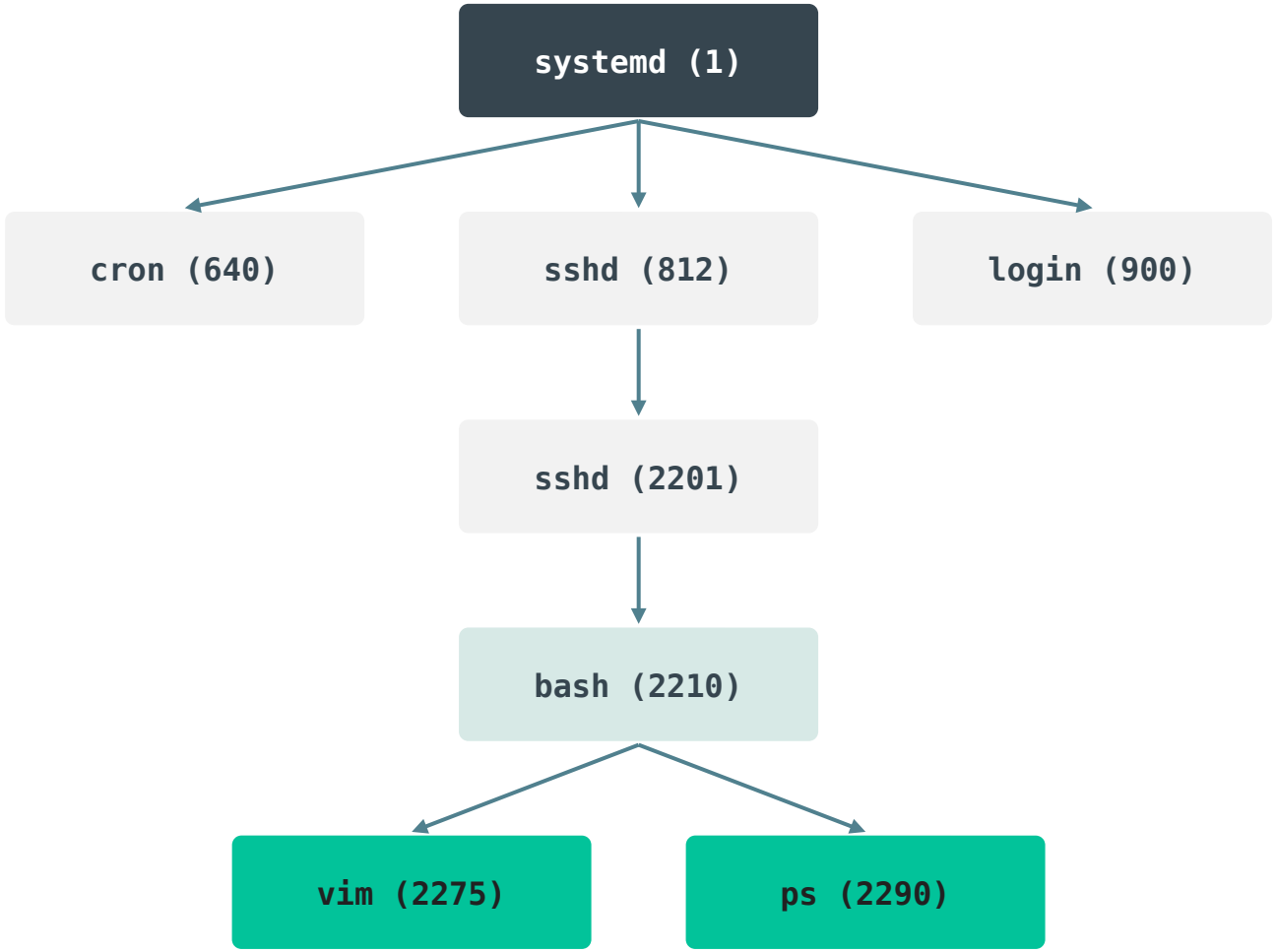
    if (pid == 0) {            /* ---- child: fork returned 0 */
        printf("child : PID %d, my parent is %d\n",
               getpid(), getppid());
        fflush(stdout);
        char list[32];         /* "parent,child" for ps -p */
        snprintf(list, sizeof list, "%d,%d", getppid(), getpid());
        execlp("ps", "ps", "-o", "pid,ppid,stat,comm",
               "-p", list, NULL);   /* replaces this program */
        exit(127);             /* reached only if exec failed */
    }
    int status;                /* ---- parent */
    waitpid(pid, &status, 0);  /* sleep until child ends */
    printf("parent: child %d finished, exit status %d\n",
           pid, WEXITSTATUS(status));
}
/* full program, with error checks: code/fork_exec_wait.c */
```

What to observe

- Same if-test, two answers: the child sees 0, the parent the child's PID.
- ps lists PID 32269 as "ps": exec kept the PID.

```
$ ./fork_exec_wait
parent: my PID is 32268
child : PID 32269, my parent is 32268
  PID  PPID STAT COMMAND
32268 32251 S    fork_exec_wait
32269 32268 R    ps
parent: child 32269 finished,
        exit status 0
```

The Process Tree and Process IDs



typical Linux server (PIDs illustrative)

```
$ bash -c 'sleep 5 & sleep 5 & pstree -p $$'
bash(6297)-+-pstree(6300)
           |-sleep(6298)
           `-sleep(6299)
```

Process identifiers

- Every process has a unique PID and knows its parent's PID (PPID).
- PID 1 (systemd or init) is started by the kernel at boot: the ancestor of all user processes.
- PIDs are reused after a process ends; the upper limit is in /proc/sys/kernel/pid_max.

Zombies and Orphans

When a process ends, its PCB entry must survive until someone collects the exit status.

Zombie

- The child has terminated, but its parent has not called wait() yet.
- Memory is freed; only the PCB entry (PID, exit status) remains.
- Many zombies can use up all PIDs.

Orphan

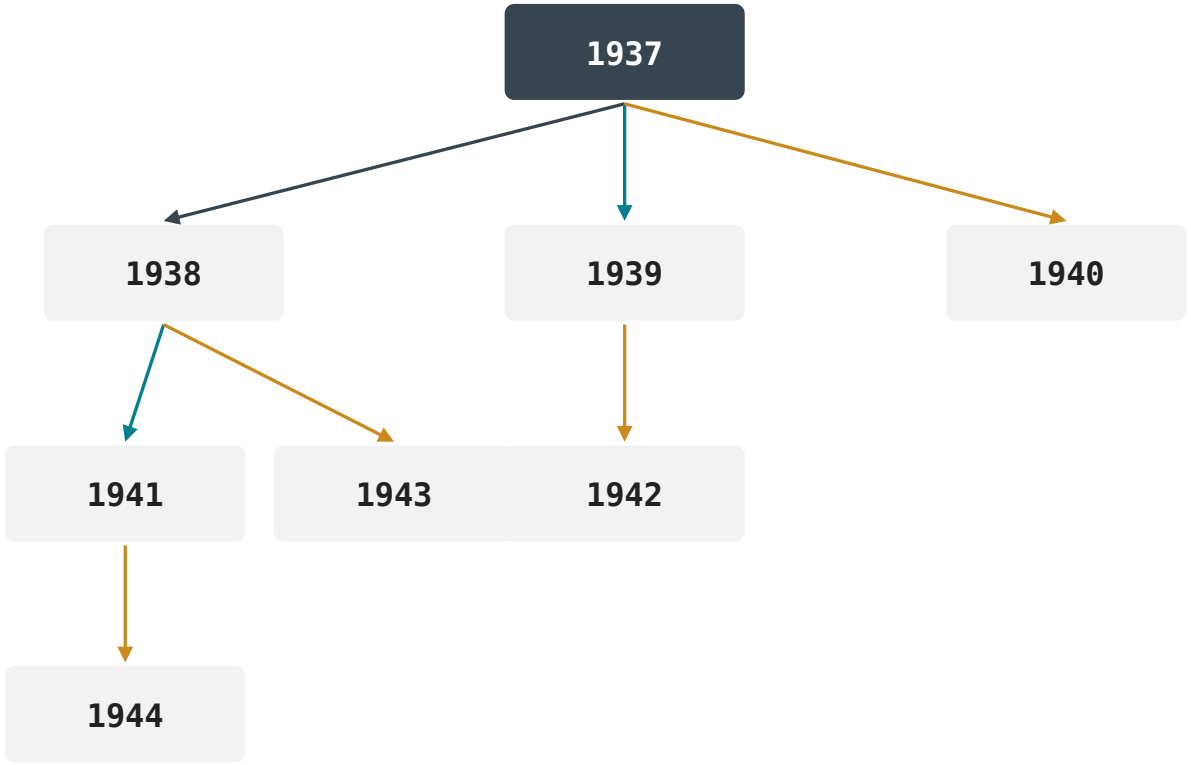
- The parent terminated first, while the child still runs.
- Linux re-parents it to PID 1 (or a "subreaper"), which calls wait() for it.
- Some systems instead kill all children (cascading termination).

```
$ sh -c 'sleep 1 & exec sleep 4' &      # then, 2 s later:
$ ps -o pid,ppid,stat,cmd -p 6385 --ppid 6385
  PID  PPID STAT CMD
 6385  6384 S    sleep 4
 6387  6385 Z    [sleep] <defunct>
```

sleep 4 never calls wait(), so its finished child stays a zombie (state Z).

EXERCISE

How Many Processes After n Calls to fork()?



arrow colour = which fork created the child: ***1st*** ***2nd*** ***3rd***

real PIDs from ./fork_count 3

```
for (int i = 0; i < n; i++)
    fork();     /* everyone forks */
printf("PID %6d  parent %6d\n",
       getpid(), getppid());
```

```
$ for n in 1 2 3 4 5 10; do
>   ./fork_count $n | wc -l; done
2  4  8  16  32  1024        (one line per process)
```

Every process doubles at each fork: after n forks there are 2^n processes in total, $2^n - 1$ of them newly created. The printf runs 2^n times.

TRY IT YOURSELF

Look at Processes on Your Own Machine

```
# Linux (lab PCs, WSL, or a virtual machine)
$ ps -ef | head            # all processes, with PPID
$ ps -o pid,ppid,stat,cmd  # your processes + state
$ pstree -p | less         # the process tree
$ cat /proc/$$/status      # the PCB of your shell
$ strace -c ls             # system calls by type
$ make && ./fork_count 4   # Week 3 code folder

# Windows
> tasklist                 # list processes
# Task Manager > Details: PID column
```

Questions to answer

- What is the PID of your shell, and who is its parent?
- How many processes are in state R right now? Why so few?
- How many voluntary context switches has your shell made so far?
- Find which of the six system-call categories ls uses most.

Week 3 — Key Takeaways

- **Services:** user-facing (UI, execution, I/O, files, communication, errors) and system-facing (allocation, accounting, protection).
- **System calls:** reached through an API and a libc wrapper; selected by a number in the system-call table; entered with a trap.
- **Parameters:** registers, a memory block, or the stack; x86-64 Linux uses rax + up to six registers.
- **Process:** a program in execution with text, data, bss, heap and stack; described by its PCB.
- **States:** new, ready, running, waiting, terminated; interrupts and system calls cause the transitions.
- **Context switch & creation:** switching is pure overhead s/(q+s); fork–exec–wait builds the process tree.

QUICK QUIZ

Check Your Understanding

1. Is printf() a system call? Explain what happens between printf() and the screen.

2. Name the three ways of passing parameters to the OS. Which one does x86-64 Linux use for write()?

3. Why can a process not go directly from waiting to running?

4. A context switch costs 10 µs and the time slice is 1 ms. What percentage of CPU time is overhead?

NEXT WEEK

Threads & CPU Scheduling Basics

Week 4 will cover:

- Threads: one process, several flows of execution
- User threads, kernel threads and multithreading models
- CPU scheduling: the CPU–I/O burst cycle and the dispatcher
- Scheduling criteria: utilisation, throughput, turnaround, waiting and response time

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz